

Component library

INFO

Lesinhoud

In dit onderdeel leer je wat CSS Frameworks zijn. Je verkent populaire frameworks en ontdekt hoe CSS Frameworks worden gebruikt binnen component libraries (Vuetify, BootstrapVue, Tailwind UI, ...). Tenslotte koppel je een component library aan je Vue-project (Vuetify).

Doelstellingen

De student(e) kan:

- Werken met een Command Line Interface (CLI) / Node Package Manager (NPM) voor het installeren, updaten en verwijderen van pakketten en modules binnen een project.
- Een single-page frontend applicatie ontwikkelen met Vue.js.
- Zelfstandig de bovenstaande technologieën combineren tot een werkende webapplicatie.
- Een CSS framework gebruiken.

▼ Inhoud

- [Wat is een CSS Framework?](#)
- [Wat is een component library?](#)
- [Vuetify installeren](#)
- [Vuetify gebruiken](#)
 - [Componenten](#)

- [Application](#)
- [Utility classes](#)
- [Vuetify + SCSS](#)

Wat is een CSS Framework?

Een CSS Framework is een verzameling van vooraf geschreven CSS-code die je helpt om het bouwen van webapplicaties te versnellen. Het biedt een set van gestandaardiseerde stijlen, componenten en layoutstructuren die ontwikkelaars kunnen gebruiken om snel en efficiënt responsieve en consistente gebruikersinterfaces te creëren.

Enkele populaire CSS Frameworks zijn:

- [Bootstrap](#)
- [Tailwind CSS](#)
 - Dit bevat geen kant-en-klare componenten, maar **utility classes** die gecombineerd kunnen worden om zelf componenten te maken.
- [Foundation](#)
- [Bulma](#)
- [Materialize](#)
 - Gebaseerd op Google's Material Design

CSS Frameworks

Binnen dit opleidingsonderdeel kijken we naar een component library waarin een CSS Framework is geïntegreerd (Vuetify met Material Design).

Wat is een component library?

In een ander hoofdstuk leer je dat het werken met componenten in Vue heel belangrijk is. Een component library is een verzameling van herbruikbare UI-componenten die je gebruikt in een specifiek frontend framework. Een component library kan verschillende UI-elementen bevatten, zoals knoppen, formulieren, navigatiebalken, modals, kaarten, ...

Een component library kan je zien als een uitbreiding van een CSS Framework, het biedt niet alleen stijlen, maar ook de bijbehorende functionaliteit en interactiviteit die nodig is om deze componenten te laten werken binnen een specifiek frontend framework.

Enkele voorbeelden van component libraries voor Vue zijn:

- [Vuetify](#) (gebaseerd op Material Design)
- [BootstrapVue](#) (gebaseerd op Bootstrap, maar dit is nog een oudere versie van Bootstrap)
- [Quasar](#) (gebaseerd op Material Design, dit biedt meer dan alleen een component library, het is een volledig ecosysteem voor het bouwen van Vue-applicaties)
- [Vuetiful](#) (gebaseerd op Tailwind CSS)
- [PrimeVue](#) (optie om `unstyled` -componenten te gebruiken, zodat er eigen stijlen kunnen worden toegepast - bijvoorbeeld door een CSS Framework zoals Tailwind CSS te gebruiken)

Vuetify

Binnen dit opleidingsonderdeel kijken we naar Vuetify als component library.

Vuetify installeren

Bekijk de [documentatie](#). Merk op dat er verschillende manieren zijn om Vuetify te installeren.

Voor dit opleidingsonderdeel kijken we naar:

- Existing projects
- npm (niet pnpm, yarn, bun).

Volg de stappen onder Existing projects, gebruik Option A om Roboto font toe te voegen.

Opdracht

Genereer een nieuw Vue-project met `npm create vue@latest`. Installeer Vuetify volgens de stappen in de documentatie.

▼ Oplossing

Na het installeren van Vuetify zal de `main.js` deze nieuwe code bevatten:

```
import { createPinia } from 'pinia';
import { createApp } from 'vue';

import { createVuetify } from 'vuetify';
import * as components from 'vuetify/components';
import * as directives from 'vuetify/directives';
import 'vuetify/styles';

import App from './App.vue';
import router from './router';

const app = createApp(App);

app.use(createPinia());
app.use(router);

const vuetify = createVuetify({
  components,
  directives,
});

app.use(vuetify);
```

```
app.mount('#app');
```

Na het installeren van het Roboto font, zal de `main.js` deze nieuwe code bevatten:

```
import { createPinia } from 'pinia';
import { createApp } from 'vue';

import { createVuetify } from 'vuetify';
import * as components from 'vuetify/components';
import * as directives from 'vuetify/directives';
import 'vuetify/styles';
```

js

```
import '@fontsource/roboto/100.css'
import '@fontsource/roboto/300.css'
import '@fontsource/roboto/400.css'
import '@fontsource/roboto/500.css'
import '@fontsource/roboto/700.css'
import '@fontsource/roboto/900.css'
import '@fontsource/roboto/100-italic.css'
import '@fontsource/roboto/300-italic.css'
import '@fontsource/roboto/400-italic.css'
import '@fontsource/roboto/500-italic.css'
import '@fontsource/roboto/700-italic.css'
import '@fontsource/roboto/900-italic.css'
```

```
import App from './App.vue';
import router from './router';
```

```
const app = createApp(App);
```

```
app.use(createPinia());
app.use(router);
```

```
const vuetify = createVuetify({
  components,
  directives,
});
```

```
app.use(vuetify);
```

```
app.mount('#app');
```

Vuetify gebruiken

Componenten

Net zoals eigen componenten, kan je componenten van het framework gebruiken in Vue. Hoe je deze gebruikt, hangt af van hoe de component library deze componenten heeft opgezet.

Bekijk de verschillende componenten in de [documentatie](#).

Opdracht

Maak een nieuwe pagina in het Vue-project op de route `/vuetify-demo`.
Gebruik minstens vijf verschillende Vuetify-componenten op deze pagina.

Application

Vuetify biedt een `v-app` component die als root component voor de applicatie dient. Dit component zorgt ervoor dat de Vuetify-stijlen correct worden toegepast en dat de layout van de applicatie goed werkt.

Bekijk de [documentatie](#) voor meer informatie over het `v-app` component en hoe dit gebruikt kan worden.

Opdracht

Pas het hoofdcomponent (`App.vue`) aan om het `v-app` component te gebruiken als root component.

Zorg dat je kan wisselen tussen het `light` - en `dark` -thema van Vuetify via een knop in de navigatiebalk.

Utility classes

Vuetify biedt verschillende utility classes die je kan gebruiken om snel stijlen toe te passen op elementen. Je kan deze utility classes gebruiken voor marges, padding, kleuren, typografie, en meer.

Bekijk de [documentatie](#) over de beschikbare utility classes in Vuetify.

Een voorbeeld is het gebruik van `d-flex` om een element **display flex** toe te kennen, of `ma-4` om een **marge all** toe te voegen van 4 eenheden (1 eenheid = 4px).

Opdracht

Gebruik minstens drie verschillende utility classes van Vuetify in de pagina die eerder gemaakt is op de route `/vuetify-demo`.

Suggesties:

- gebruik `d-flex` en `justify-center` om een element horizontaal te centreren.
- gebruik `pa-4` om padding toe te voegen aan een container.
- gebruik `mb-8` om een marge onder een element toe te voegen.

Combineren met andere CSS Frameworks

Wees voorzichtig bij het combineren van Vuetify met andere CSS Frameworks zoals Bootstrap of Tailwind CSS. Dit kan leiden tot conflicten in stijlen en onverwacht gedrag van componenten. Het is aan te raden om één CSS Framework of component library per project te gebruiken om consistentie en voorspelbaarheid te waarborgen.

De enige uitzondering is het gebruik van `unstyled` -componenten, zoals die van PrimeVue, in combinatie met een CSS Framework zoals Tailwind CSS. Dit

voorkomt conflicten omdat de componenten geen eigen stijlen hebben.

Vuetify + SCSS

Vuetify is onderliggend opgebouwd met SCSS, wat betekent dat je de mogelijkheid hebt om de standaard stijlen van Vuetify aan te passen door gebruik te maken van SCSS-variabelen en mixins.

In een ander hoofdstuk wordt de werking van SCSS uitgelegd.

Bekijk de [documentatie](#) voor meer informatie over het aanpassen van Vuetify met SCSS.

Niet verplicht

Het is geen vereiste om Vuetify aan te passen via SCSS in dit opleidingsonderdeel, maar het is goed om te weten dat deze mogelijkheid bestaat voor toekomstige projecten.

Previous page
[Oefeningen](#)

Next page
[Oefeningen](#)